



School of Computer Science and Engineering

Fall Semester 2024-25

CAT I

SLOT: E1

Programme Name & Branch: B.E Computer Science and Engineering / SCOPE

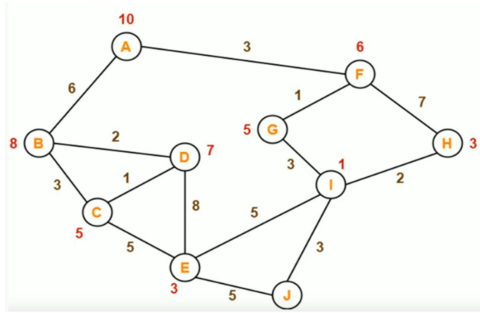
Course Name & Code: BSCE306L / Artificial Intelligence

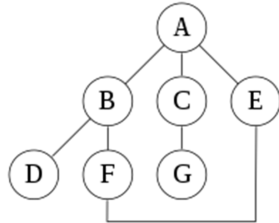
Class Number (s): VL2024250101496

Faculty Name (s): Dr. R. Naga Priyadarsini

Exam Duration: 90 Min.

Maximum Marks: 50

Q.No.	Question	Max Marks
1.	Illustrate the PEAS components and classify the environment type for the provided scenarios. a) Customer browsing an online shopping website b) Chatbot customer service c) Home automation system	10
2.	a) Consider a scenario where a bank wants to enhance its fraud detection system. Justify how AI can identify fraudulent transactions, prevent money laundering and improve risk assessment. (5 marks) b) Illustrate how the subfield of AI Expert system works and where is it typically applied? (5 marks)	10
3.	Apply A* search algorithm on the provided graph and determine the optimal path from node B to node J with the minimum total cost. Also, provide the pseudocode for the A* search algorithm as part of your solution. 	10

4.	a) Enumerate classical “8 queen problem”. Describe the state space for this problem and give the solution. (5 marks) b) Apply depth-first search and iterative deepening search techniques to traverse a graph, starting at node A and aiming to reach node G. (5 marks) 	10
5.	A manufacturing plant aims to create an optimal production schedule that reduces production time and efficiently utilizes resources. Discuss how hill climbing could be applied to determine a feasible schedule, highlighting potential obstacles. Also, illustrate how simulated annealing can be used to overcome suboptimal solutions and achieve a better schedule.	10

School of Computer Science and Engineering

Fall Semester 2024-25

CAT I – ANSWER KEY

SLOT: E1

**Programme Name & Branch: B. Tech. Computer Science and Engineering /
SCOPE**

Course Name & Code: BSCE306L / Artificial Intelligence

S.No.	Answers	Marks
1.	Scenario A: Customer Browsing an Online Shopping Website - Performance Measure: - User satisfaction (how happy the customer is with their experience), - Speed of page load (how quickly the website responds), - Ease of navigation (how easily the customer can find what they need), - Relevance of product recommendations (how closely recommendations match customer interests), - Conversion rate (whether the customer makes a purchase). - Environment: - Partially Observable: The system doesn't know everything about the customer's preferences or intentions. - Stochastic: The behavior of the customer is unpredictable and can vary widely. - Dynamic: The customer's interests and the website content can change over time. - Continuous: The interaction happens in real-time without distinct phases. - Multi-Agent: Multiple customers may be interacting with the system simultaneously. - Actuators: - Displaying products, updating shopping cart, providing search results, offering recommendations. - Sensors: - Mouse clicks, typing inputs, scroll movements, time spent on pages, previous purchase history. Scenario B: Chatbot Customer Service - Performance Measure: - Accuracy of responses (how correctly the chatbot answers questions), - Speed of response (how quickly the chatbot replies), - Customer satisfaction (how pleased the customer is with the	10

	interaction), - Resolution rate (whether the customer's issue is resolved). 2. Environment: - Partially Observable: The chatbot may not have full knowledge of the user's issue or the context of the conversation. - Stochastic: The user's inputs and queries may be unpredictable and vary widely. - Dynamic: The conversation evolves with each interaction between the user and the chatbot. - Sequential: The conversation follows a sequence of steps, with each response depending on the previous input. - Multi-Agent: The user and possibly other chatbots or human agents might be involved in the interaction. 3. Actuators: - Textual responses, triggering actions (e.g., providing a link, escalating to a human agent). 4. Sensors: - User text input, sentiment analysis, context from previous interactions. Scenario C: Home Automation System 1. Performance Measure: - Energy efficiency (minimizing energy consumption), - User comfort (maintaining a comfortable living environment), - Security (ensuring the home is safe), - Responsiveness to user commands (how quickly the system reacts to user inputs). 2. Environment: - Fully Observable: The system can monitor and control all aspects of the home environment through various sensors. - Deterministic: The system's actions have predictable outcomes, such as turning on lights or adjusting the thermostat. - Dynamic: The environment changes over time, such as temperature fluctuations or movement in the house. - Continuous: The system continuously monitors and adjusts the environment without interruption. - Single-Agent: Typically, only one system is managing the home environment, though it may interface with various devices. 3. Actuators: - Adjusting thermostat settings, turning lights on/off, locking/unlocking doors, controlling appliances. 4. Sensors: - Temperature sensors, motion detectors, cameras, door/window sensors, voice commands.	
2.	A: AI in Enhancing a Bank's Fraud Detection System 1. Identifying Fraudulent Transactions Pattern Recognition: AI systems can be trained using historical data to recognize patterns of normal transactions and detect anomalies that might indicate fraud. Machine learning algorithms, especially those based on supervised learning, can classify transactions as either	10

	legitimate or fraudulent based on features like transaction amount, location, time, and user behavior. • Real-time Monitoring: AI can continuously monitor transactions in real-time, identifying suspicious activity immediately. This is particularly useful for detecting anomalies that deviate from a user's normal behavior. • Behavioral Analysis: AI can analyze user behavior over time. For example, if a transaction occurs in a location far from the user's usual area or involves an unusually large amount, the system can flag it for further review. 2. Preventing Money Laundering • Link Analysis: AI can analyze and map the connections between various transactions to identify complex money-laundering schemes that involve multiple parties and transactions. It can detect patterns such as layering (splitting transactions to obscure their origin) and integration (reintroducing laundered money into the financial system). • Risk Scoring: AI can assess the risk associated with transactions by evaluating factors such as the origin and destination of funds, the parties involved, and the transaction's purpose. High-risk transactions can be flagged for additional scrutiny. • Natural Language Processing (NLP): NLP can analyze large volumes of unstructured data, such as customer communication and documentation, to identify potential red flags associated with money laundering activities. 3. Improving Risk Assessment • Predictive Analytics: AI can use historical data to predict potential risks associated with certain customers, accounts, or transactions. This allows banks to take proactive measures to mitigate risks before they escalate. • Customer Profiling: AI can build detailed profiles of customers based on their transaction history, behaviors, and other relevant data. These profiles help in assessing the risk level of each customer more accurately. • Scenario Analysis: AI can simulate various risk scenarios to evaluate the impact of potential fraud or financial crimes on the bank, allowing for better preparation and response strategies. B. How Expert Systems Work • Knowledge Base: An expert system contains a knowledge base that stores facts and rules about a specific domain. This knowledge is usually gathered from human experts in the field. • Inference Engine: The inference engine is the core component that applies logical rules to the knowledge base to derive conclusions or make decisions. It works by reasoning through the rules and facts, much like a human expert would. • User Interface: The user interface allows users to interact with the expert system. Users input queries or data, and the system provides advice, recommendations, or solutions based on its reasoning process. 2. Applications of Expert Systems • Medical Diagnosis: Expert systems are widely used in healthcare to assist doctors in diagnosing diseases by analyzing patient data and comparing it with known symptoms and conditions stored in the system. • Financial Services: In finance, expert systems help in making investment	
--	--	--

	decisions, assessing credit risks, and managing portfolios by analyzing market trends, financial data, and risk factors. • Manufacturing: Expert systems are used in manufacturing for quality control, fault diagnosis, and process optimization by monitoring production processes and identifying potential issues before they escalate. • Legal Advice: Legal expert systems provide guidance on legal matters, helping lawyers or clients navigate complex legal issues by analyzing relevant laws, regulations, and case precedents. Illustration: Expert systems simulate the decision-making ability of a human expert. They consist of a knowledge base with domain-specific information and an inference engine that applies logical reasoning to solve problems or provide recommendations. They are typically applied in domains requiring specialized knowledge, where consistent and reliable decision-making is critical.	
3.	Path: $B \rightarrow C \rightarrow E \rightarrow J$ Cost: $(3+5+5) + 0 = 13$ 1. Initialize: set OPEN=[s], CLOSED=[], $g(s)=0$, $f(s)=h(s)$ 2. Fail: If OPEN=[], then terminate and fail 3. Select: Select a state with minimum cost ,n, from OPEN and CLOSED 4. Terminate: If $n \in G$ then terminate with success and return $f(s)$ 5. Expand: For each successors , m of n For each successor, m, insert m in OPEN only if if $m \notin [OPEN \cup CLOSED]$ set $g(m) = g[n] + C[n,m]$ Set $f(m) = g(m) + h(n)$ if $m \in [OPEN \cup CLOSED]$ set $g(m) = \min \{g[m], g(n) + C[n,m]\}$ Set $f(m) = g(m) + h(m)$ If $f[m]$ has decreased and $m \in CLOSED$ move m to OPEN 6. Loop: Goto step 2	10
4.a	Initial: 8x8 chess board Actions: horizontally, vertically, or diagonally. Goal cost: 8 queens should be placed in the chessboard such that no queen can attack another Path cost: Cumulative cost of each action when placing a queen in the board (usually assumed to be 1 per action).	5
4.b	DFS: $A \rightarrow B \rightarrow D \rightarrow F \rightarrow E \rightarrow C \rightarrow G$ IDS: Level 0: A Level 1: $A \rightarrow B \rightarrow C \rightarrow E$ Level 2: $A \rightarrow B \rightarrow D \rightarrow F \rightarrow C \rightarrow G$	5
5.	Hill Climbing Hill climbing is a simple optimization algorithm that iteratively moves towards a higher-valued solution. In the context of manufacturing scheduling, the "value" could be a combination of factors like production time, resource utilization, and	10

	cost. Steps: 1. Generate an initial schedule 2. Evaluate the schedule 3. Explore neighbors 4. Select the best neighbor 5. Repeat Potential Obstacles: 1. Local optima 2. Plateaus Simulated Annealing Simulated annealing is a more robust optimization technique that allows for occasional "uphill" moves, helping to avoid local optima. It is inspired by the annealing process in metallurgy, where a material is heated and then slowly cooled to reach a low-energy state. Steps: 1. Generate an initial schedule. 2. Evaluate the schedule. 3. Explore neighbors. 4. Select the best neighbor. 5. Accept or reject: ○ If the new schedule has a higher value, accept it. ○ If the new schedule has a lower value, accept it with a probability that decreases over time (simulating cooling). This probability is determined by a temperature parameter. 6. Update temperature: Gradually decrease the temperature parameter. 7. Repeat steps 3-6 until the temperature reaches a predefined minimum or a satisfactory solution is found. Advantages of Simulated Annealing: 1. Escaping local optima 2. Flexibility Let's consider a simple manufacturing plant with 5 tasks (A, B, C, D, E) and 2 machines. Our goal is to minimize the total completion time (makespan). Initial schedule (randomly generated): Machine 1: A(3) -> C(4) -> E(2) Machine 2: B(5) -> D(3) Makespan: 9 units Hill Climbing: 1. Swap C and E on Machine 1 Machine 1: A(3) -> E(2) -> C(4) Machine 2: B(5) -> D(3) Makespan: 9 units (no improvement) 2. Move D to Machine 1 Machine 1: A(3) -> D(3) -> E(2) Machine 2: B(5)	
--	--	--

	-> C(4) Makespan: 9 units (no improvement) Hill climbing gets stuck in this local optimum. Simulated Annealing: 1. Same as hill climbing step 1 (no improvement, but accepted with some probability) 2. Move B to Machine 1 Machine 1: A(3) -> B(5) Machine 2: C(4) -> D(3) -> E(2) Makespan: 9 units (no improvement, but accepted with some probability) 3. Swap A and C Machine 1: C(4) -> B(5) Machine 2: A(3) -> D(3) -> E(2) Makespan: 9 units (no improvement, but accepted with lower probability) 4. Move E to Machine 1 Machine 1: C(4) -> E(2) -> B(5) Machine 2: A(3) -> D(3) Makespan: 11 units (worse, but accepted with low probability) 5. Swap B and E Machine 1: C(4) -> B(5) Machine 2: A(3) -> D(3) -> E(2) Makespan: 8 units (improvement!) Simulated annealing found a better solution by temporarily accepting worse schedules, allowing it to escape the local optimum that trapped hill climbing.	
--	---	--